

Grounded Control Interfaces for Contact-Rich Robot Agents

Feedback Programs, Position–Force Policies, and Joint Residuals

Research Proposal I · Zimo Wen

September 2026

Abstract

Robot agents can describe a goal and invoke manipulation skills while still failing during contact, obstruction, or precise placement. One potential bottleneck is the action interface: a position target expresses where to move but does not fully express how firmly to make contact, when to yield, or how to compensate for local dynamics. This proposal studies whether task-conditioned control representations improve contact-rich manipulation under matched sensing and interaction budgets. Building on Robot-RSI, it compares three approaches: generating executable feedback programs, predicting task-space position–force and impedance parameters, and learning bounded joint-space residual control. The intended paper will establish when a richer control interface improves both task completion and physical interaction quality, and when simpler feedback is sufficient. Experiments separate the contributions of sensing, action representation, control frequency, and training data, with contact quality and deployment cost evaluated alongside success.

1. Motivation, Research Gap, and Paper Hypothesis

The starting point. Robot-RSI already produces executable skills and numerical motion commands. Its LIBERO end-effector increment skill uses servo control and motion solving to produce joint increments and gripper commands. The research question is therefore which additional control variables a model should choose, from which observations, and at what update rate. It is not the introduction of numerical actions into an otherwise purely semantic agent.

The inspected Flexiv adapter exposes joint state, joint torque, external wrench, and gripper-force parameters. A contact-descent handler stops when the measured force change crosses a threshold. These capabilities provide an entry point, but sensing contact and maintaining a desired contact force are different behaviors. The inspected Cartesian-motion wrapper mainly passes pose and velocity and retains hardware-interface checks. Gripper closure force, contact detection, and a six-dimensional end-effector wrench must remain distinct experimental variables.

Motivating tasks. Placing an object on a surface of uncertain height requires a controlled transition from free motion to contact. Pushing along a surface requires maintaining normal contact while progressing tangentially. Pulling a resistant drawer requires distinguishing friction, approach misalignment, and grasp slip. A feedback program, task-space impedance, or joint residual might solve these cases; lower-level output is not assumed to be universally better.

Central hypothesis. A task-conditioned interface that expresses contact goals and feedback can improve manipulation more efficiently than position commands alone, while joint-level learning is useful only when task-space feedback cannot compensate for the relevant error. This is a self-contained paper about **the appropriate action abstraction for robot agents**. Approach B, task-conditioned position–force and impedance control, is the primary method. Feedback programs provide a structured baseline; joint residuals test whether still lower-level output is necessary.

Prior work and intended novelty. Code as Policies already includes feedback programs and impedance-control examples. Diffusion Policy and π_0 provide continuous action-generation foundations. The proposed increment is to ground control structure or parameters in task intent and actual contact evidence, then identify their generalization boundaries. Adding a force field to an API or using a larger action network alone is insufficient. The main claim must be supported by unseen objects and dynamics, matched observations, and measurable physical improvements.

2. Approach A: Executable Local Feedback Programs

Representation and execution. A model generates a program with branches, loops, and termination conditions that reads visual error, proprioception, grasp state, and available force measurements. A program might approach the visible target, reduce speed after contact, and change a local search direction when progress stalls while lateral force rises. The language model supplies inspectable logic; a local runtime executes feedback at a declared rate without requiring a remote language-model call for every small correction.

Generation contract. Expose typed primitives for reading errors, setting short-horizon pose references, observing contact, changing modes, and terminating motion. Each program declares its coordinate frame, action bounds, maximum duration, update period, and exit conditions. Static validation checks types, unbounded loops, and interface consistency. A single control owner applies commands. These checks establish executability, not physical correctness; candidate programs must still be evaluated inside a simulator feedback loop.

Learning and adaptation. Development traces connect scene conditions, candidate programs, and observed responses. The agent generates and revises programs using development feedback and independently verified outcomes. A key comparison asks whether a fixed feedback structure transfers by updating perception-derived parameters, or whether regenerating the entire program is necessary. Test-time models, prompts, and libraries are frozen. Local program generation may occur within an episode, but test outcomes do not become persistent capabilities for later evaluation tasks.

Mechanism test. Compare fixed skills, generated open-loop programs, handcrafted feedback programs, and generated feedback programs. Open- and closed-loop variants share approach trajectories and primitives as far as possible. The difference should concern observation-conditioned corrections rather than access to stronger tools. If the agent only selects a known template, the contribution is task-conditioned selection and transfer, not discovery of a new control law. Measure recovery from local errors, success, and model-call reduction.

3. Approach B: Position–Force and Impedance Policies

Action representation. Predict $a_t = (\Delta x, \Delta R, f^*, K, D, m)$: pose increments, desired contact force, stiffness, damping, and contact mode. A contact frame specifies normal and tangential directions. Selection matrices assign position control and force control to compatible axes, avoiding contradictory constraints on the same degree of freedom. Units, frames, timestamps, and update intervals are explicit in every action packet.

Local realization. A controller uses pose, velocity, and force error to track these references. For example, a normal-direction admittance loop can adjust displacement from $f^* - f_t$ while tangential motion follows a path. The policy updates references at a lower rate than the device-supported feedback loop. Parameter amplitude and slew-rate limits constrain changes. Contact establishment, maintenance, and separation have distinct transitions. Positive stiffness is an engineering constraint, not a proof of overall closed-loop stability.

Training. Collect successful and unsuccessful development interactions from verified control templates and learn the mapping from instruction and observation to control parameters. Compare template-conditioned prediction with a continuous action-generation head. Training can supervise parameter trajectories and execution outcomes, but model selection depends on physical performance on validation tasks. Sensor bias, frame conversion, and filter delay are recorded so that gravity or inertial changes are not automatically mistaken for contact events.

Expected advantage and limitations. This representation uses low-dimensional physical structure and may need less data than learning all joint actions. Its success can nevertheless depend on contact-direction estimates and template coverage. Compare fixed parameters, per-task manual tuning, and observation-conditioned parameters to isolate online adaptation. A model that outputs one constant parameter set has learned parameter selection, not necessarily state-dependent contact control.

4. Approach C: Bounded Joint-Space Residual Control

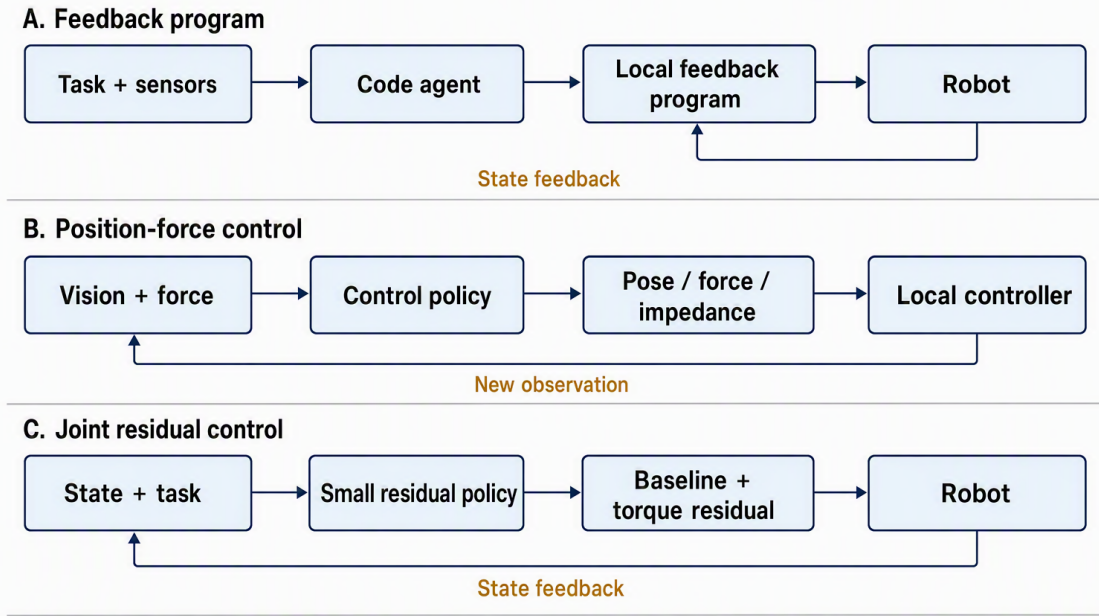
Formulation. Retain a validated baseline controller and use a small local action network to predict a corrective joint torque from proprioception, force sensing, and task context:

$$\tau_t = \tau_{\text{base}}(s_t) + \text{clip}(\delta\tau_\theta(s_t, g, h_t), -b, b).$$

Residual amplitude, slew rate, and contact-power limits are fixed after validation. The local network executes in the control path; language reasoning provides slower task context. Token-by-token generation of high-frequency torque by a general language model is not required.

Learning and interfaces. In a simulator that actually supports torque commands, establish baseline behavior before learning residual corrections through demonstrations or reinforcement learning. Randomize load, friction, delay, and sensor bias to discourage compensation for a single simulator defect. The actor only receives deployable observations. Hidden dynamics may define evaluation strata or offline analyses but never become actor inputs. Hardware transfer requires checking control frequency, communication delay, and baseline-controller equivalence.

Action Interfaces for Robot Agents



Three alternative action abstractions. Matched observations and budgets.

Figure 1: Candidate control interfaces. The three rows produce feedback programs, task-space references, or joint residuals. In row B, robot execution is included in the controller path and observations return from robot sensors. GPT-image-2 conceptual diagram; no measured result is depicted.

Discriminating comparison. Compare no residual, end-effector residuals, impedance adjustment, and joint torque residuals with matched sensing, training data, and approximately matched capacity. If lower-level output helps only for specific disturbances, report that boundary. Retain unstable and saturated trajectories. Clipping a residual constrains execution but does not establish reliability or prove that the learned component caused a successful task.

5. Experimental Design and Evaluation Panels

Standard versus contact evaluation. The standard 120-task LIBERO short panel tests whether the new execution component preserves basic capability. It reports frozen task-level pass@10 by spatial/object/goal subset and task/object/swap/lan suite. A separate contact-control extension evaluates force and torque interfaces, documenting changes to controllers, sensors, and action spaces. Improvements in this extension do not count as standard-benchmark gains.

Main contact panel. Use three task families: contact placement, surface pushing, and drawer interaction. Four object or geometry configurations per family give 12 frozen task instances. Development and validation use different objects, layouts, and physical conditions. Three predetermined disturbance levels and five initial seeds per task yield 180 episodes per method. Each task varies quantities with a clear physical interpretation, such as friction, resistance, or surface height.

First vary friction, load, surface location, and action delay separately; then evaluate held-out combinations, such as surface-height error combined with changed friction. Development verifies that the perturbation range remains solvable using a known effective controller. Greater resistance must not be confused with physical impossibility. Slip and contact-loss criteria use task-appropriate windows and report detection errors.

Comparison	Action and feedback	Matched conditions
Position control	Pose or joint increments	Vision, proprioception, and available force
Contact-stop baseline	Position motion with force threshold	Calibration and execution limits
Handcrafted feedback	Fixed program and tuned controller	Primitives and observation delay
Generated feedback	Conditions and recovery branches	Primitives and model budget
Learned contact policy	Pose, force, and impedance	Training data and contact limits
Joint residual	Baseline plus bounded torque	Observations and approximate capacity

Separating representation from sensing. The main comparison supplies the same force observations to all methods, followed by a no-force ablation. This separates access to better information from the ability to express different control. Record control-update rate, action horizon, and low-level limits. Where architectures cannot be identical, report parameter count, training data, training compute, and deployment latency, with capacity- or latency-matched auxiliary comparisons.

Physical-robot validation. If the Flexiv continuous-force interface and logging chain are verified, use the three matched task families with three physical configurations and ten randomized trials each: an initial 90 trials per method. Calibration and model selection occur before formal trials; training and test objects remain separate. Include joint residuals only if the actual torque interface and controller have been validated. Otherwise, retain torque conclusions as simulation-only and avoid presenting them as successful sim-to-real transfer.

Prespecified endpoints. Final task success is the main behavioral endpoint. Select one task-appropriate contact-quality endpoint before testing, such as peak placement force or force-tracking error during sustained pushing. The panel sizes above are proposed starting points; validation variance determines whether additional task diversity is necessary to support the intended effect-size claim.

6. Planned Experiments and Expected Interpretations

Experiment A: action abstraction. Compare position interfaces, generated feedback, contact parameters, and joint residuals under identical visible observations. Evaluate task success together with interaction quality so that faster completion with a larger impact is not summarized as an unqualified improvement. Retain force and action traces for failed episodes as well as successful ones.

Physical metrics. For contact establishment, report peak force, contact impulse, overshoot, and settling time. During contact maintenance, report force-tracking RMSE, contact-loss fraction, and slip. During motion, report position error, completion time, and abnormal oscillation. Force error is computed only during intervals with a defined target force; including free-space intervals would artificially reduce the average. Report physical units by task before applying any preregistered cross-task aggregation.

Experiment B: contact modes and compliance. Hold the policy structure fixed and remove adaptive stiffness, adaptive damping, mode transitions, or short-term feedback in turn. Compare force-only, compliance-only, and joint prediction. Align parameter traces with contact phases to identify redundant outputs or constant parameter selection. Evaluate whether parameters change appropriately when resistance increases, rather than merely whether the final task succeeds.

Experiment C: whether generated feedback is used. Keep program structure and total action budget fixed while delaying or freezing one feedback channel, or removing the recovery branch relevant to a validated local disturbance. Test whether the program changes behavior after a contact anomaly. Bound full-program regeneration so that results do not rely on unlimited replanning. Attribute failures to perception, conditions, primitives, or interfaces before labeling them language-reasoning failures.

Experiment D: what residuals compensate. Change loads and friction and compare learned residuals with equal-magnitude random residuals, end-effector residuals, and a better-tuned baseline. If the learned residual mainly cancels a constant gravity-compensation error, calibration may be a simpler explanation. Introduce delay and sensor bias to test amplification of high-frequency noise. Report all protective interventions and unstable traces.

Possible result	Defensible interpretation
Feedback improves recovery; contact references reduce force peaks	The interfaces address different errors; explain behavioral and physical gains separately.
Impedance matches torque residuals with less data	Low-dimensional structure suffices; joint-level output is not generally necessary.
Residuals help only under unseen loads	Supports a specific compensation role; examine regression under ordinary conditions.
Success rises alongside oscillation or protection triggers	Overall control quality has not necessarily improved.
Force sensing helps, but action interfaces perform similarly	Sensing may dominate; narrow the representation claim.

Statistics and cost. Use matched seeds and bootstrap intervals clustered by task instance. Randomize hardware method order within trial blocks to reduce thermal and equipment-state effects. Record model calls, tokens, perception/action/recovery time, training volume, and protection-trigger rate. Label secondary analyses explicitly to avoid selecting favorable curves after testing.

7. Feasibility, Risks, and Expected Paper Contribution

An actual interface is a prerequisite. The inspected LIBERO path uses joint increments. Force and torque experiments require implemented, functionally validated controllers. Minimal checks cover free-space tracking, contact with a known surface, coordinate transforms, sensor offsets, mode changes, and command latency. Hardware checks combine SDK semantics with functional trials. These establish that the research variable exists; a working controller is not itself evidence that learning improved behavior.

Stability boundaries. Generated programs, positive impedance parameters, and bounded residuals do not guarantee stability under arbitrary contact. Analyze appropriate local models or passivity assumptions where possible and measure behavior outside those assumptions. Report saturation frequency and the actions actually executed whenever predictions frequently hit limits, allowing readers to distinguish model behavior from the baseline controller’s protective behavior.

Simulation-to-real uncertainty. Friction, contact stiffness, filtering, and tool parameters can differ between simulation and hardware. Document the training randomization range, validation choices, and real test conditions separately. Success with one object does not establish contact generalization. Matched and unseen conditions both require evaluation, and non-transferring failure modes remain part of the result.

Outcome accounting. Only final simulator or independent-harness verdicts count as task success. Infrastructure interruptions, implementation errors, and protection events are reported separately. A method that often fails to execute cannot claim superior reliability from the success rate of its remaining valid episodes. Main tables include scheduled instances, valid verdicts, success/failure counts, infrastructure records, and implementation-error coverage. Preserve videos, force curves, trajectories, diagnostic runs, and exact manifests.

Expected contribution. The paper should establish which errors are addressed by feedback programs, task-space contact references, and joint residuals; introduce a grounded mapping from task intent and contact evidence to executable control; and evaluate that mapping on unseen objects and dynamics. Contact quality, inference delay, and data cost are release criteria alongside success. The contribution is a supported account of when a richer interface is useful, not the fact that an agent can output lower-level numbers.

Independence from the other papers. This study uses a fixed skill library and declared control models. It does not require a new responsiveness algorithm or persistent GPT review. The candidate approaches share a task and logging protocol but receive separate mechanism analyses. The proposed sample counts and anticipated outcomes are experimental designs, not completed results. Training loss remains a diagnostic; trained policies must be tested in simulation on matched and held-out tasks.

References

- [1] Code as Policies: Language Model Programs for Embodied Control. 2022. <https://arxiv.org/abs/2209.07753>
- [2] Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. 2023. <https://arxiv.org/abs/2303.04137>
- [3] π_0 : A Vision-Language-Action Flow Model for General Robot Control. 2024. <https://arxiv.org/abs/2410.24164>
- [4] Octo: An Open-Source Generalist Robot Policy. 2024. <https://arxiv.org/abs/2405.12213>
- [5] Noematrix Team. RoboRSI research preview. 2026. <https://lab.noematrix.ai/blog/2-roborsi-research-preview/>

Responsive Robot Agents

Event-Triggered Execution, Asynchronous Actions, and Predictive Control

Research Proposal II · Zimo Wen

September 2026

Abstract

A robot agent reasons from observations that are already in the past, while previously issued actions may continue during inference and communication. Target movement, grasp slip, obstruction, or an instruction change can therefore invalidate an otherwise reasonable plan before it finishes executing. This proposal studies responsiveness as the time required for an environmental event to produce an effective change in physical action. It develops and compares three approaches: event-triggered preemptible skills, delay-aware asynchronous action chunking, and predictive control with uncertainty. Robot-RSI provides the initial execution infrastructure. Controlled dynamic perturbations will separate sensing, detection, reasoning, scheduling, and actuation delays, while jointly evaluating recovery, final success, and computational cost. The intended paper asks when execution-time adaptation yields benefits beyond faster inference or more frequent observation.

1. Research Problem and Standalone Paper Claim

A concrete failure. A robot reaches for a cup, but the cup moves during the reach. Producing the next instruction faster does not ensure that the old motion has stopped. A new instruction based on the earlier image may simply execute an outdated action sooner. Grasp slip requires a different response, and a revised user instruction may require a new goal. These events differ in urgency and in what constitutes successful correction.

Starting from Robot-RSI. Some existing motion skills already contain local feedback, and the Flexiv contact-descent routine can detect force internally and stop. These capabilities must be distinguished from task-level adaptation during execution. In the inspected sidecar, motion, state reads, and stop requests share a request lock, so an external observation or stop can wait for the current handler. This finding concerns the software request path, not independent hardware emergency-stop behavior. The research target is timely incorporation of new evidence into execution.

Definition. Let t_e denote the physical event time and t_a the time when a corrective action begins to take effect. End-to-end response latency is $L = t_a - t_e$. The evaluator records t_e without announcing it to the policy. Recovery time and final task success are separate endpoints: immediate stopping may shorten a response measure without completing the task. A text statement that the agent has responded is not an actuation measurement.

Central claim to test. Managing action validity and commitment under changing observations can improve disturbed-task success and response-tail latency at matched compute, beyond merely increasing observation frequency or reducing model latency. Event-triggered execution, asynchronous chunking, and predictive control provide three technical routes to this claim. Each has an appropriate matched baseline; they need not be combined to demonstrate an effect.

Prior work and hypotheses. Real-Time Chunking (RTC) already studies asynchronous action-chunk execution. The proposed increment concerns invalidation by environmental change, measured physical response, and its relationship to final success. H1: genuine preemption improves recovery at matched sensing frequency. H2: delay- and state-conditioned action commitment improves over fixed chunking at matched inference budget. H3: calibrated short-horizon prediction improves delayed control, while prediction errors can cause regression. The paper must identify these boundaries instead of assuming that asynchronous or predictive systems are inherently more responsive.

Primary method and scope. The main method will be event-triggered, preemptible execution in Approach A. Asynchronous chunking is a matched execution baseline and possible integration path; predictive control is an extension. The paper centers on whether observable changes invalidate actions and produce timely, effective recovery under matched sensing and compute.

2. Approach A: Event-Triggered, Preemptible Skills

Events and validity conditions. A skill declares observable preconditions: its target remains within a valid region, grasp state is unchanged, or the visible path remains compatible with the plan. A timestamped sensor stream feeds an event detector using change magnitude, confidence, and task phase. Small deviations may remain within the local controller’s responsibility. Larger changes trigger recovery or replanning, preventing every noisy image from interrupting stable motion.

Preemption protocol. Action packets carry a skill identifier, plan version, source-observation timestamp, expiry time, and cancellation marker. When an event invalidates a packet, the runtime revokes unexecuted commands and a single control owner stops or changes the motion and acknowledges the transition. A later model result based on an obsolete plan or observation must be revalidated before use. Cancelling a software wait does not stop the robot; logs must distinguish cancellation requested, accepted, and physically observed.

Recovery. Prefer local, verifiable responses such as slowing, holding, retracting briefly, or observing again. Invoke a slower Planner when the goal or program assumptions have changed. Debouncing, hysteresis, and cooldown reduce repeated triggers, but cooldown can also conceal important events and must be evaluated. Compare fixed thresholds with an event score learned on development data and frozen at test time. Ground-truth event times may support offline analyses but never drive the online detector.

Core comparison. Compare end-of-skill inspection, periodic inspection, and event-triggered execution with identical sensing rates. Measure false triggers, missed events, revoked commands, and stale actions that still execute. This separates better detection from actual preemptibility. Implement priority cancellation through the control owner or its queue, rather than allowing multiple threads to switch robot modes concurrently.

3. Approach B: Delay-Aware Asynchronous Action Chunking

The tradeoff. A continuous-action model predicts H future actions at once. Longer chunks reduce inference stalls but extend the influence of stale observations. Short chunks update more often, yet inference can outlast their execution and create gaps or discontinuities. The proposed method overlaps next-chunk inference with current execution and adapts the committed prefix to measured delay and scene change.

Alignment. With control interval Δt and estimated inference-plus-transfer delay $\hat{L}$, an initial commitment estimate is $p \approx \lceil \hat{L} / \Delta t \rceil$, refined by actions already executed and the device’s irrevocable buffer. A new chunk must join the actual execution state, not the state when inference was requested. Recent delay distributions support the estimate, with explicit behavior under unexpected tail latency.

Conditioning on change. A fixed prefix may be unnecessarily conservative in static scenes and too slow under target motion. Use target velocity, contact changes, and uncertainty to adjust the horizon and overlap. If the prefix becomes invalid, replace it at the cancellation boundaries supported by the device. Velocity or acceleration continuity constraints reduce abrupt transitions. The method must report any movement restrictions introduced by this commitment strategy.

Research increment. RTC is the main baseline. Compare fixed asynchronous chunks, delay-only adaptation, state-change-only adaptation, and their combination using the same action model and weights. This keeps the research centered on execution. If state-aware commitment does not improve final success or response tails, the conclusion should be limited to the measured continuity or utilization benefit.

4. Approach C: Predictive State Updates and Control

Predicting the state at actuation. Instead of using the latest target position directly, estimate the state when an action will take effect. Inputs include observation history, proprioception, candidate actions, and measured delay. A short-horizon model predicts future state and uncertainty. Receding-horizon optimization selects an action sequence, executes a prefix, and corrects predictions using new observations. Constant-velocity extrapolation and simple dynamics are explicit baselines before introducing a learned action-conditioned model.

Objective. Balance expected task cost, uncertainty, and control changes:

$$\min_{a_{t:t+H}} \mathbb{E}[C_{\text{task}}] + \lambda_U C_{\text{uncertainty}} + \lambda_A C_{\text{action}}.$$

High uncertainty may shorten the horizon, request another observation, or favor a conservative local action. The predictor uses permitted observations only. Simulator future states can measure offline prediction errors but are never supplied to the deployable planner.

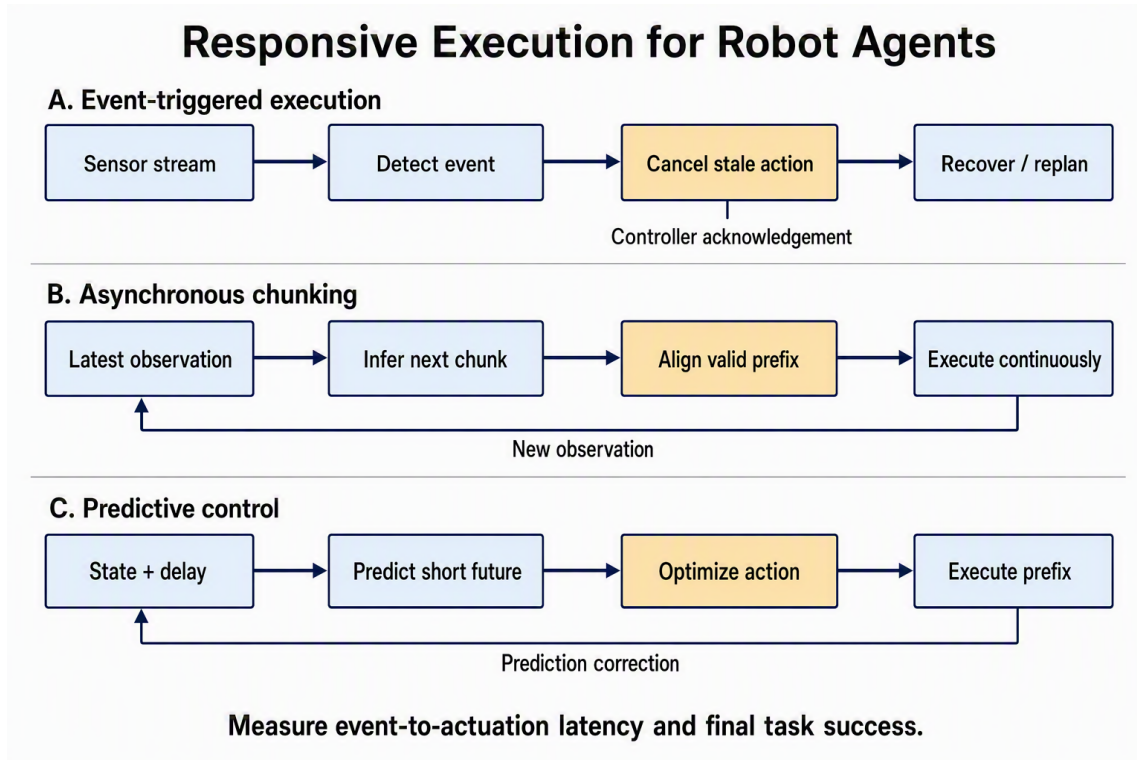


Figure 1: Independent pipelines for event-triggered execution, asynchronous chunking, and prediction. GPT-image-2 conceptual diagram. The evaluation target is physical correction and eventual task completion, not the time at which the model finishes generating a response.

Mechanism tests. Compare smooth target motion, sudden direction changes, brief occlusion, and contact-condition changes. Prediction may help under smooth motion and fail under abrupt changes; uncertainty calibration and observation correction should explain the boundary. Report both prediction accuracy and control performance. More accurate images or states do not automatically imply better actions, and excessive conservatism can reduce completion. Hardware timestamps must be aligned before making millisecond-scale claims.

5. Experimental Design: Dynamic Tasks and Delay Interventions

Perturbation panel. Extend LIBERO short tasks with four task-appropriate event classes: target movement, grasp/contact change, visible path-condition change, and instruction revision. Events occur before completion of the relevant action and leave a feasible task. For example, arbitrary teleportation of an already grasped target is not used as a natural-motion perturbation. An independent evaluator injects events, and policies discover them through declared sensing or instruction channels.

Proposed scale. Select 24 task–event pairs, covering each event class across multiple manipulation tasks. Three disturbance strengths, four delay conditions, and five seeds give 1,440 episodes per method. Pairs sharing an underlying task are not statistically independent. Separate development and validation panels determine solvability, magnitude, and timing before the test list is frozen. Adjust sample requirements using development variance rather than observed test effects.

Delay interventions. An initial design adds 0, 100, 300, or 600 ms of inference delay while recording actual total latency. Separate experiments add delay to sensing transfer or action submission; these are distinct interventions. A tail-latency panel compares distributions with similar means and different p95 values. If baseline inference already greatly exceeds the proposed bins, revise them against measured control and task timescales on validation, then freeze them.

Method	Update and execution	Mechanism isolated
Synchronous skill baseline	Observe after skill completion	Existing task-level feedback
Periodic inspection	Matched sensing, fixed update period	Value of observation frequency
Event preemption	Validity checking and acknowledged cancellation	Interruption and recovery
Short synchronous chunks	Frequent inference before execution	Resource cost of updates
RTC baseline	Asynchronous fixed-model chunks	Continuity under inference delay
State-aware chunking	Adaptive commitment and invalidation	Joint response to delay and change
Predictive control	Delay compensation and correction	Value of anticipating future state

Controlling confounds. Event methods share perception models and rates; chunking methods share action-model weights; predictive methods are compared under matched compute and movement limits. Record sampling rates, queue length, inference calls, and cumulative compute. Giving one method more GPUs or denser sensing does not establish a scheduling contribution. Report matched-budget comparisons and success–latency–cost tradeoffs where exact matching is impossible.

Benchmark boundaries. Report dynamic extensions separately from the standard 120-task short panel. The latter uses task-level pass@10 with spatial/object/goal and task/object/swap/lan breakdowns. Dynamic main tables report success on fixed perturbation instances and latency distributions. Exact manifests bind tasks, seeds, event trajectories, trigger rules, latency samples, and method versions. Preserve evidence and successful-seed resume protection. Cross-method trials are independent comparisons, while an unchanged method does not rerun an already successful task/seed configuration.

6. Planned Experiments: From Events to Recovery

Timing decomposition. Record physical event time t_e , the first sample containing the event t_s , detection t_d , corrective decision readiness t_p , controller acceptance t_c , and physical action change t_a . Report $t_s - t_e$, $t_d - t_s$, $t_p - t_d$, $t_c - t_p$, and $t_a - t_c$ where defined. A continuously updating method may have no discrete detection event; compare common measurable stages without inventing timestamps.

Defining effective response. Prespecify criteria by event: motion toward the changed target, grasp maintenance or recovery after slip, slowing/stopping/rerouting after obstruction, or ceasing actions that contradict a revised instruction. Validation freezes these rules, and the evaluator applies them offline. Observable physical change establishes response; a Planner declaration does not. Final task success remains an independent harness verdict.

Experiment A: event-triggered mechanisms. At matched sensing rates, ablate preemption, expiry checks, and learned versus fixed event thresholds. Measure false triggers, missed events, failed acknowledgements, and stale commands. Introduce noise and occlusion to test whether hysteresis generalizes. Accurate detection without physical correction points to the execution path; fast stopping without successful recovery supports only a narrower claim.

Experiment B: asynchronous execution and tails. Vary the mean, variance, and spikes of inference latency, then examine chunk length, committed prefix, state-change threshold, and transition smoothing. Primary measures are disturbed-task success, response p95, and continuity; report p50, p99, and control gaps as secondary measures. Strong p99 claims require enough events. Measure velocity and acceleration changes so that aggressive motions cannot masquerade as an unqualified responsiveness gain.

Experiment C: whether prediction improves control. Remove delay compensation, uncertainty penalties, and observation correction separately. Compare constant-velocity and learned prediction for smooth motion and sudden turns. Analyze relationships between prediction error and task failure. Any future-state upper-bound analysis remains offline, in a separate table, with no future ground truth exposed to the evaluated agent.

Experiment D: responsiveness versus cost. At fixed call or compute budgets, compare success, response p95, tokens, and wall time. Event triggering may save unnecessary model calls in static scenes. Asynchrony may improve utilization while increasing cumulative compute. Prediction may be more accurate but too slow to help. Evaluate end-to-end cost rather than only reduced time waiting for a language model.

Possible observation	Appropriate conclusion
Faster response and higher disturbed-task success	Supports effective responsiveness after matching sensing and compute.
Faster stopping but slower recovery or more failures	Improves the stop path, not necessarily dynamic manipulation.
Smooth chunks still execute stale actions after abrupt changes	Continuity improves while validity management remains a bottleneck.
Prediction helps smooth motion but hurts sudden turns	State the motion boundary and analyze correction and uncertainty.
Frequent periodic checks perform equally well	The event method's distinct benefit may be computational savings.

7. Statistical Protocol, Risks, and Paper Contribution

Statistics. Use paired bootstrap intervals clustered by underlying task, sharing event trajectories and latency samples across methods. Preregister final disturbed-task success and response p95; detection, recovery, and compute are explanatory metrics. Episodes that never respond cannot silently disappear from latency statistics. Report nonresponse rate, timeout threshold, and conditional latency quantiles; use censoring-aware analysis where appropriate. Otherwise, a method responding only on easy cases could appear artificially fast.

Physical execution evidence. A submitted command may wait while the robot executes an older buffer. Validate response through proprioception, controller acknowledgements, and synchronized timestamps. Quantify inter-device clock error. Keep simulator physical time separate from wall time, especially in accelerated simulation. Record frame production, arrival, and model-consumption times separately to expose observation staleness.

Principal risks. Event detectors can overfit task-specific signals, requiring task-family holdouts. Repeated interruptions can destabilize otherwise effective local control, requiring explicit cancellation boundaries and smooth transitions. Predictive models can fail abruptly, requiring calibration and regression analysis. The three approaches require different model or execution interfaces, so use matched baselines for each rather than forcing an unfair single ranking.

Outcome and efficiency reporting. Only simulator or independent-harness verdicts count as task success. Provider, transport, image, resource, and interrupted records are excluded from valid-verdict task denominators and reported separately. Also report implementation failures, nonresponses, protection events, and incomplete coverage. Log prompt/completion/total tokens, VLM and unmetered calls, wall time, and perception/action/recovery time. Preserve videos, trajectories, failures, and diagnostic logs. Learned models must be evaluated on matched and held-out tasks; lower training loss is not proof of improved response.

Expected contribution. This paper should establish an execution-time account of responsiveness: when event-triggered validity checks, adaptive action commitment, or short-horizon prediction can improve behavior under environmental change. Its methodological contribution is a comparison with matched sensing and compute; its evaluation contribution is the separation of event time, physical correction, recovery, and final completion. The strongest supported claim must depend on actual closed-loop behavior.

Independence. The study can use fixed skills or a frozen VLA and does not depend on learning a new force controller or persistent GPT review. Candidate methods receive independent ablations under the common question. Task counts, perturbations, and delay bins are proposed experimental settings to freeze after validation, not completed measurements or guaranteed gains.

References

- [1] Real-Time Execution of Action Chunking Flow Policies. 2025. <https://arxiv.org/abs/2506.07339>
- [2] Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. 2023. <https://arxiv.org/abs/2303.04137>
- [3] π_0 : A Vision-Language-Action Flow Model for General Robot Control. 2024. <https://arxiv.org/abs/2410.24164>
- [4] Code as Policies: Language Model Programs for Embodied Control. 2022. <https://arxiv.org/abs/2209.07753>
- [5] Noematrix Team. RoboRSI research preview. 2026. <https://lab.noematrix.ai/blog/2-roborsi-research-preview/>

GPT-Reviewed VLA

Keep the Policy or Correct Its End-Effector Action

Research Proposal III · Zimo Wen

September 2026

Abstract

A vision–language–action policy can execute familiar manipulation efficiently, while a general-purpose GPT model can inspect task context and identify errors that the policy may miss. This proposal studies a deployed hybrid agent that retains both components. The VLA proposes an action chunk, and GPT chooses between two execution branches: keep the original action, or construct a bounded end-effector (EEF) correction through Code as Policy and audit it before execution. The same GPT model can perform selection, correction, and review in separate calls. The central question is whether selective correction improves task success without damaging actions the VLA would already execute correctly. We also propose a benchmark with natural rollouts and private, paired checkpoint continuations to separate branch selection, correction quality, unnecessary intervention, and inference cost. GPT remains active during evaluation and deployment; removing it is not the objective.

1. Research Question and the Intended Paper

Deployment mechanism. A VLA proposes behavior from an instruction and current observations. GPT reviews this proposal and the visible scene. If it is appropriate, execution retains the original action. If a local geometric change is needed, GPT writes an EEF correction, reviews that correction, and submits the approved result. After execution, new observations re-enter the loop. This is a persistent review-and-correction system, rather than a training-only teacher whose role disappears after fine-tuning.

Why two branches matter. A reviewer that always intervenes can damage an already capable VLA. A reviewer that always accepts preserves the VLA but cannot rescue mistakes. For example, the proposed grasp may already align with a cup, or it may need a small lateral or orientation adjustment. The research problem is to identify the cases where preserving the proposal is preferable and to generate a useful correction when it is not. Decision quality and correction quality must both be measured.

EEF correction, not policy-weight modification. Here EEF denotes end-effector pose or motion commands. GPT changes a bounded part of the current action trajectory, using explicit position/orientation offsets, local waypoints, or a short feedback program. The main study freezes VLA weights and retains existing low-level controllers. GPT does not introduce a learned force controller or update the VLA’s parameters at each review. The effect on the executed policy comes from selecting and modifying actions at runtime.

Central hypothesis. A persistent GPT reviewer can improve a frozen VLA’s closed-loop success by applying a small number of useful EEF corrections, while retaining most actions that are already effective. The paper should show a favorable success–intervention–latency tradeoff compared with always keeping, always correcting, and matched heuristic gates. It should also identify conditions in which review or self-audit is unreliable.

Relationship to the other proposals and prior work. The first paper studies which control quantities the agent should output; the second studies when changed observations can affect execution. This paper studies **which VLA proposals to trust and how to correct the selected ones**. Code as Policies provides executable programs, while VLA policies provide nominal actions. DAgger-style aggregation is a possible later extension, but is not the main deployment mechanism or evaluation target. A broader literature comparison is needed before asserting a first-of-its-kind contribution.

2. Method: Binary Review with an Audited EEF Correction

Visible input. Let $A_t = \pi_\theta(o_t, g, h_t)$ be the VLA’s proposed short action sequence. GPT receives the instruction, allowed images and proprioception, bounded visible history, and an inspectable representation of A_t . Where a VLA outputs joint commands, a declared adapter may express their intended EEF motion using robot kinematics. It must not inspect hidden object poses or simulator contact truth. The candidate remains associated with its observation timestamp, action version, and coordinate convention.

Binary execution rule. The reviewer produces $b_t \in \{\text{KEEP}, \text{CORRECT}\}$ and a concise, observable justification. For an approved correction program p_t :

$$A_t^{\text{exec}} = \begin{cases} A_t, & b_t = \text{KEEP}, \\ C(A_t, p_t, o_t), & b_t = \text{CORRECT}. \end{cases}$$

C applies an explicitly bounded EEF edit and translates it to the existing action interface. KEEP means the unchanged valid proposal, not a newly generated replacement trajectory. CORRECT edits the declared segment and records what changed. These remain the two task-action branches; waiting for a fresh observation or reporting an invalid proposal is execution management, not an undisclosed third controller.

Correction and audit. In the CORRECT branch, GPT creates a candidate containing a frame, translation/rotation or waypoints, maximum duration, and observable completion conditions. The same GPT snapshot reviews the candidate against the original proposal and scene in a separate call. A deterministic interface check independently validates types, finite values, action bounds, and supported calls. Allow at most a preregistered number of revisions. Rejected or expired corrections are not silently executed, and invalid proposals count against completion and cost. GPT approval is not a simulator success verdict.

3. Three Technical Approaches to the Correction Branch

Approach A: geometric EEF edits. GPT predicts a small translation and rotation offset for a selected VLA segment, with explicit tool- or world-frame semantics. Apply the edit to a trajectory reference and preserve the rest of the valid proposal. This is the simplest interface for testing whether the reviewer can identify misalignment and useful correction direction. Compare with matched fixed or perception-derived offsets; correct units and orientation composition are essential.

Approach B: bounded Code-as-Policy repair. GPT produces a short local program using declared EEF primitives, for example re-observe, move to a corrected approach pose, and resume the skill. The program may use local feedback but has a fixed duration and revision budget. It cannot replace the task with an unrestricted new plan. Retaining the VLA proposal, affected segment, and handoff rule makes the correction inspectable and separates local repair from full GPT takeover.

Approach C: prediction-assisted review. Present GPT with a standardized kinematic preview or observation-conditioned prediction of the original and corrected short motions, including uncertainty. GPT selects or revises the correction using only deployable evidence. Compare with the same reviewer without previews, matching call budgets. A learned preview can be wrong; future simulator states are never shown to the reviewer. This approach tests whether action consequences improve binary judgment beyond inspecting the current image and action numbers.

4. Pipeline, State Validity, and Runtime Boundaries

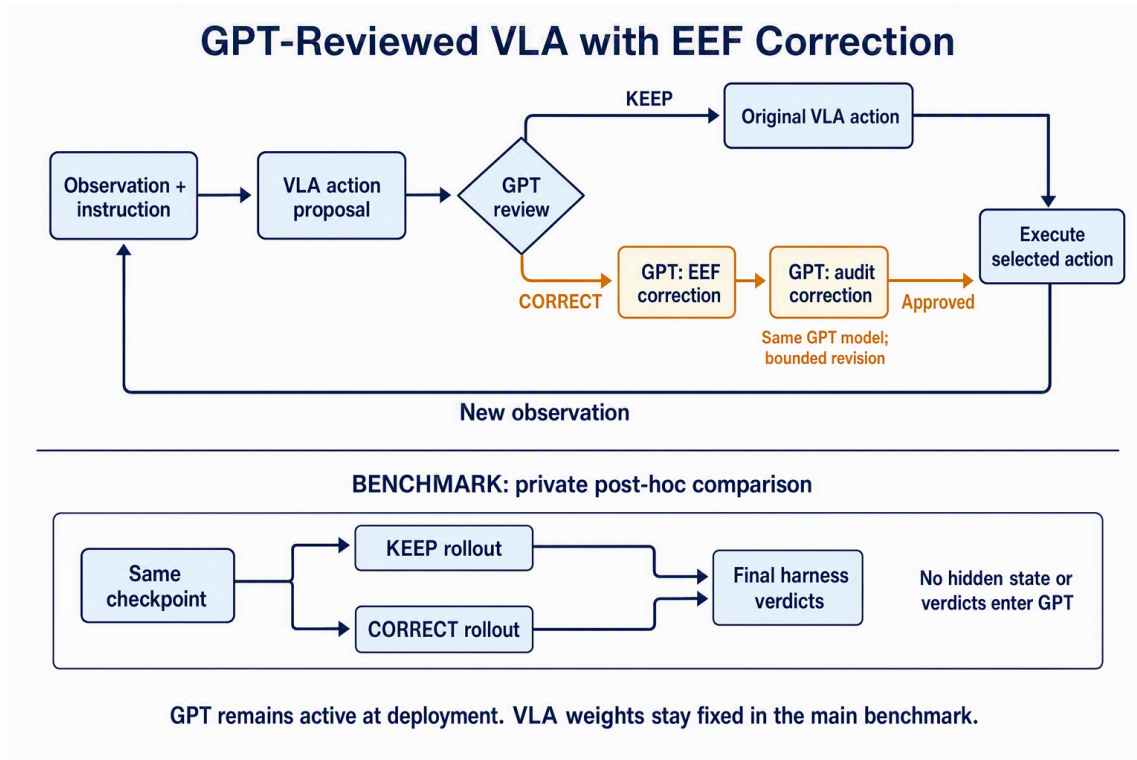


Figure 1: GPT remains in the deployment loop. Review selects KEEP or CORRECT; GPT-generated EEF corrections are audited before execution. The separate benchmark panel compares outcomes privately and never returns hidden state or branch verdicts to GPT. GPT-image-2 conceptual diagram, not a measured result.

One GPT model, several roles. Selection, correction generation, and audit can use the same fixed GPT model and version. They are distinct calls with bounded inputs, not independent sources of evidence. Correlated mistakes and confirmation bias are therefore explicit risks. Compare a single-pass choice with a second-pass audit, while holding total token or call budgets fixed. A model name, including a chosen GPT release, is a configuration to record rather than a guarantee of review accuracy.

Preventing stale edits. Before applying an EEF correction, check the observation and proposal versions. After any intervention, obtain a fresh observation and re-query the VLA; replaying an old, geometrically invalid suffix is prohibited. During review, use a declared hold or validated executable prefix. Environmental evolution and review latency are included in real-time evaluation. A paused-simulation diagnostic must be labeled separately and cannot establish deployment responsiveness.

Existing implementation entry points. Robot-RSI exposes relative EEF motion, pose observation, and servo-based correction paths. These support a concrete adapter but do not establish that arbitrary VLA chunks can already be edited correctly. Validate action scaling, frames, quaternion conventions, gripper semantics, cancellation, and action ownership. Hidden target positions, simulator predicates, and evaluator-owned checkpoints remain outside GPT and generated-code access.

5. Benchmark Design: Selection, Correction, and Full Episodes

Proposed benchmark. We propose *VLA Review Benchmark* as a working title. It evaluates the deployed VLA+GPT system, with frozen VLA and reviewer configurations. The two tracks below distinguish local decision quality from complete robot behavior. The benchmark is initially built on LIBERO short tasks; its checkpoint and disturbance extensions are reported separately from standard task-level scores.

Track A: paired checkpoint continuations. Retain checkpoints from natural VLA rollouts using an evaluator-only mechanism. At a checkpoint, present the same permitted observation and original VLA proposal to each reviewer. Execute the KEEP continuation and the submitted CORRECT continuation in separate simulator branches with matched initial state and exogenous randomness, using the same frozen VLA for subsequent continuation. The agent cannot inspect checkpoint internals or other branch outcomes. Endpoint labels come from final harness verdicts, not a GPT judgment that the EEf pose looks better.

Let $Y_K, Y_C \in \{0, 1\}$ be final success for the unchanged proposal and the submitted correction. The four cases $(0, 1)$, $(1, 0)$, $(1, 1)$, and $(0, 0)$ respectively indicate a beneficial correction, a harmful correction, equal successful outcomes, and unresolved failure. These labels describe the evaluated candidate and continuation, not a universal oracle declaring that a state is correctable by any possible method. For stochastic environments, estimate outcome probabilities over matched continuation seeds rather than relying on one binary result.

Lock the review decision before evaluating either branch. For KEEP decisions, obtain a counterfactual correction in a separate diagnostic call without allowing the decision to change. This avoids measuring correction quality only on states the reviewer selected. Its extra generation and branching costs belong to benchmark construction, not deployed inference cost. Track A isolates checkpoint decisions; Track B includes actual review delay and environmental evolution.

Track B: closed-loop episodes. Deploy the complete reviewer with its real decisions, repeated observations, correction audits, and latency. This tests distribution shift caused by its own interventions. A good checkpoint score does not ensure a good multi-step hybrid policy. Include unperturbed tasks where the VLA already works, local geometric errors that EEf edits may repair, and cases outside the allowed correction envelope. The last group prevents the benchmark from rewarding plausible but ineffective edits; failure remains a legitimate outcome.

Proposed panel and splits. An initial diagnostic test panel contains 24 held-out task-condition pairs with 10 retained checkpoints each: 240 checkpoints. Add a closed-loop panel of the same 24 pairs with five independent initial seeds: 120 episodes per method. Development and validation use different task families and objects. These counts are starting designs, adjusted using development variance; near-duplicate task recipes stay within one split. Release a naturally sampled panel and a separately labeled, stratified stress panel rather than silently balancing outcomes to favor correction.

Configuration and fairness. Freeze model snapshots, prompts, correction bounds, review frequency, revision limits, preview methods, and budgets before testing. Record model-specific sampled checkpoints; do not assume one VLA’s failure states represent another VLA. Compare additional VLA backbones and reviewer models on declared tracks. All algorithms receive the same allowed sensing, action access, and wall-time accounting. The standard 120-task LIBERO panel additionally reports pass@10 by spatial/object/goal and task/object/swap/lan.

6. Planned Experiments and Benchmark Metrics

Main comparisons. Use VLA-only execution, GPT review constrained to KEEP, unconditional GPT EEF correction, a heuristic gate with the same GPT corrector, and the full GPT gate–corrector–audit system. Add a GPT gate with a fixed geometric repair to isolate judgment from generation. An unrestricted GPT takeover is a separate capability/cost reference, not an equal-action-space baseline. Compare geometric edits, bounded programs, and prediction-assisted review under matched budgets.

Decision metrics. On paired continuations, report the rate of choosing CORRECT when $(Y_K, Y_C) = (0, 1)$ and KEEP when $(1, 0)$. Report the unresolved and equal-outcome fractions separately. In equal-success cases, unnecessary intervention incurs cost even when success is preserved. A paired candidate regret can be computed as $\max(Y_K, Y_C) - Y_b$, where Y_b is the selected branch outcome. This is a hindsight diagnostic, never an online selection signal or proof of optimality beyond those two candidates.

Correction metrics. Measure accepted program validity, actual EEF displacement and orientation change, correction duration, local recovery evidence, and final success change. Report harm on checkpoints where the original VLA succeeds. Rejected candidates, audit revisions, correction-limit saturation, and unexecuted proposals remain visible. Improvement from moving the EEF closer to a visually chosen point is not sufficient if it reduces final task success.

Experiment A: preserve versus rescue. On the natural panel, compare the hybrid’s paired success gain and intervention rate against the VLA. On the stress panel, separate local misalignment, grasp/placement mistakes, observation ambiguity, and failures beyond the correction envelope. A useful system should rescue some VLA failures while limiting newly introduced failures. Report both terms instead of only net improvement.

Experiment B: is audit useful? Compare no second-pass audit, same-model audit, and deterministic interface checks alone. Hold total model budgets fixed in one comparison and allow actual incurred costs in another. Evaluate whether the second pass catches bad edits or merely repeats the original rationale. GPT self-consistency is not independent verification; only executed outcomes establish intervention benefit.

Experiment C: correction scope and state freshness. Vary EEF edit magnitude, orientation freedom, repair duration, and revision count. Compare fresh re-observation and VLA re-query after edits with a labeled diagnostic that reuses the old proposal suffix. Separately inject review delay to quantify stale-action failure. This paper uses latency as a cost and failure factor; designing a new low-latency scheduler remains the second proposal’s objective.

Possible result	Defensible interpretation
More rescued failures than damaged successes, with bounded cost	Supports selective deployed correction over the frozen VLA.
Always-correct performs equally well	Binary review has not established an independent advantage.
Audit raises call cost without reducing harmful edits	The extra review pass is not justified under the measured budget.
Checkpoint gains disappear in closed-loop episodes	Intervention-induced state shift or stale handoffs remain unresolved.
Small geometric edits match generated programs	Simple correction may suffice; narrow the Code-as-Policy claim.

7. Statistical Protocol, Risks, and Expected Contribution

Primary evaluation. Preregister closed-loop task-success improvement at a fixed review/correction budget, with the harmful-intervention rate on baseline-successful checkpoints as a key secondary endpoint. Use paired seeds and intervals clustered by underlying task; repeated checkpoints from one rollout are not independent tasks. Report uncertainty for candidate-pair labels and candidate regret. Never return test-branch outcomes to the reviewer or use them to tune its gate. Benchmark construction and test outcome assignment remain evaluator-side operations.

Efficiency and reliability. Report prompt/completion/total tokens, VLM calls and unmetered calls, review latency, EEF correction time, recovery time, and total wall time per episode. Compare success–intervention–cost curves rather than assuming every extra review is valuable. Distinguish natural sampling from any balanced stress set. Count success only from the final simulator or independent harness. Provider, transport, image, resource, and interrupted records are excluded from valid-verdict task denominators and reported separately; implementation errors, rejection exhaustion, and missing coverage are also reported.

Principal failure modes. A reviewer can hallucinate a geometric error, confuse coordinate frames, or overrule a correct VLA. The same GPT used for generation and audit can reinforce its own mistake. A bounded correction can be locally plausible but incompatible with the next policy state. Sensing ambiguity can make reliable branch selection impossible. Each failure class needs retained images, visible rationale, action receipts, and final verdicts; hidden reasoning and simulator state are never exposed to the evaluated agent. Preserve original, corrected, failed, rejected, and diagnostic runs.

Why this is a benchmark contribution. Existing task success alone cannot reveal whether a reviewer knew when to intervene, whether its correction helped, or whether extra calls merely compensated for a poor base policy. Paired candidate continuations plus complete hybrid rollouts expose these distinctions. The benchmark should report candidate-dependent labels, natural prevalence, cross-VLA transfer, and latency conditions. It must not present privileged hindsight selection as a deployable competitor.

Optional learning extension. Correction traces may later support DAGger-style aggregation or VLA fine-tuning. If studied, evaluate the updated VLA *with GPT still active* under the same deployed architecture, and separate training gains from runtime correction gains. A GPT-free ablation can quantify component contribution, but removing GPT is neither the objective nor a requirement of this proposal. The main paper can be completed with frozen policy weights.

Expected paper claim. A persistent GPT reviewer can selectively retain VLA actions or improve them through audited EEF corrections, with measurable gains and identifiable limits. The method contribution is the binary review-and-correction interface; the experimental contribution is a benchmark that separates preserving correct behavior, repairing mistakes, and unnecessary or harmful intervention. This claim is independent of the first paper’s control representation and the second paper’s execution-timing mechanism.

References

- [1] Code as Policies: Language Model Programs for Embodied Control. 2022. <https://arxiv.org/abs/2209.07753>
- [2] π_0 : A Vision-Language-Action Flow Model for General Robot Control. 2024. <https://arxiv.org/abs/2410.24164>
- [3] Real-Time Execution of Action Chunking Flow Policies. 2025. <https://arxiv.org/abs/2506.07339>
- [4] Ross, Gordon, and Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. 2011. <https://arxiv.org/abs/1011.0686>
- [5] Noematrix Team. RoboRSI research preview. 2026. <https://lab.noematrix.ai/blog/2-roborsi-research-preview/>